Lab Manual

Programming for Artificial Intelligence (Python)

Faculty of Software Engineering

SUPERIOR UNIVERSITY LAHORE PAKISTAN

List of Experiments:

Lab. No	Experiment Names
1	Introduction to Artificial Intelligence Tools Python Basics
2	Python Basics and File Handling Introduction to Machine Learning
3	State Space Search and Problem Solving
4	Advanced Search Techniques Prolog Basics
5	Prolog Problem Solving Prolog for AI
6	Prolog for Expert Systems
7	Prolog and Machine Learning
8	Prolog Applications
9	Advanced NLP Concepts
10	NLP Applications
11	Generative AI
12	Hands-on with Generative AI
13	Flask Framework Basics
14	Flask Integration with AI Flask Project Workshop
15	Project Development Final Project Presentations

Lab 1

Introduction to Artificial Intelligence Tools:

Overview of Artificial Intelligence Tools

- Artificial Intelligence (AI) tools are software applications that enable machines to simulate human intelligence.
- These tools help in automating tasks such as decision-making, pattern recognition, problem-solving, and language understanding.
- AI tools can be categorized based on their application areas, such as machine learning, natural language processing (NLP), computer vision, and expert systems.
- Some commonly used AI tools include TensorFlow, PyTorch, OpenCV, IBM Watson, and ChatGPT.

Mapping of Tools with AI Paradigms

- **Machine Learning (ML):** TensorFlow, PyTorch, Scikit-learn
- **Natural Language Processing (NLP):** OpenAI's GPT, IBM Watson, spaCy
- **Computer Vision:** OpenCV, YOLO, Google Vision AI
- **Expert Systems:** Prolog, CLIPS
- **Robotics & Automation:** ROS (Robot Operating System), Blue Prism

Python Basics: Syntax, Functions, Control Structures, Loops

- **Syntax:** Python uses indentation instead of braces to define code blocks. It has a simple and readable syntax.
- **Functions:** Functions are defined using the def keyword. Example:

```
def greet(name):
    return "Hello, " + name
print(greet("Alice"))
```

- **Control Structures:** Python supports if, elif, and else statements for decision-making. Example:

```
x = 10
if x > 5:
    print("Greater than 5")
else:
    print("Less than or equal to 5")
```

- **Loops:** Python provides for and while loops for iteration. Example:

```
for i in range(5):
    print(i)
```

Data Structures in Python: Lists, Tuples, Sets, Dictionaries

- **Lists:** Ordered, mutable collections of elements. Example:

```
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")
print(fruits)
```

- **Tuples:** Ordered, immutable collections. Example:

```
coordinates = (10, 20)
print(coordinates[0])
```

- **Sets:** Unordered, unique elements. Example:

```
unique_numbers = {1, 2, 3, 3, 4}
print(unique_numbers)  # Output: {1, 2, 3, 4}
```

- **Dictionaries:** Key-value pairs. Example:

```
student = {"name": "Alice", "age": 20}
print(student["name"])
```

Lab 1 - Task

Task 1; Kaggle Competition: House Price Prediction.
(https://www.kaggle.com/competitions/home-data-for-ml-course)

Lab 2

Python Basics and File Handling

Classes, Inheritance, and Modules in Python

- **Classes:** Python supports object-oriented programming (OOP) through classes, which act as blueprints for creating objects.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def display_info(self):
        return f"{self.brand} {self.model}"

car1 = Car("Toyota", "Corolla")
print(car1.display_info())  # Output: Toyota Corolla
```

- **Inheritance:** Enables a class to derive properties and methods from another class, promoting code reusability.

```
class ElectricCar(Car):
    def __init__(self, brand, model, battery_capacity):
        super().__init__(brand, model)
        self.battery_capacity = battery_capacity

    def battery_info(self):
        return f"Battery Capacity: {self.battery_capacity} kWh"

tesla = ElectricCar("Tesla", "Model S", 100)
print(tesla.display_info())  # Output: Tesla Model S
print(tesla.battery_info())  # Output: Battery Capacity: 100 kWh
```

- **Modules:** Python modules allow the organization of code into separate files for better maintainability and reusability.

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"

# main.py
import mymodule
print(mymodule.greet("Alice"))  # Output: Hello, Alice!
```

File Reading and Data Preprocessing in Python

- **Reading Files:** Python provides built-in functions for reading and writing files.

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

- **Writing Files:**

```
with open("output.txt", "w") as file:
    file.write("Hello, world!")
```

- **Data Preprocessing:** Data preprocessing is essential in machine learning for handling missing values, normalization, and feature extraction.

```
import pandas as pd

df = pd.read_csv("data.csv")  # Load dataset
df.fillna(df.mean(), inplace=True)
# Fill missing values with column mean

print(df.head())  # Display first few rows
```

Introduction to Machine Learning Models:

- **Naïve Bayes:** A probabilistic classifier based on Bayes' theorem, often used for spam detection and text classification.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.2)

model = GaussianNB()
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

- **Decision Trees:** A tree-based model used for classification and regression.

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

Lab 2 - Task

Task 1; Kaggle Competition: Spaceship Titanic
(https://www.kaggle.com/competitions/spaceship-titanic)

- Download the dataset from Kaggle.
- Perform exploratory data analysis (EDA) and preprocessing.
- Train a classification model to predict survival status.
- Evaluate and improve model performance.

Lab 3

Introduction to State Space and Search Algorithms

- **State Space Representation:**
  - A **state space** is a collection of all possible states that a problem can have.
  - A problem is defined by an **initial state**, **goal state**, and a set of **rules** or **actions** that can transition from one state to another.
  - The solution is found by searching through the state space.

- **Search Algorithms:**
  - **Uninformed Search (Blind Search):** These algorithms explore the search space without using problem-specific knowledge. Examples:
    - **Depth-First Search (DFS)** – Explores as deep as possible before backtracking.
    - **Breadth-First Search (BFS)** – Explores all neighbors before moving deeper.

  - **Informed Search (Heuristic Search):** Uses problem-specific knowledge to guide the search. Examples:
    - *A Search** – Uses a heuristic function to find an optimal path.
    - **Greedy Best-First Search** – Selects the most promising node based on heuristics.

Water Jug Problem: Problem Representation and Solution

- The **Water Jug Problem** is a classic AI problem that involves two jugs of different capacities and aims to measure an exact amount of water using a set of allowed operations.

- **State Representation:** A state is represented as (x, y), where:
  - x is the amount of water in Jug 1.
  - y is the amount of water in Jug 2.

- **Example Problem:**
  - Given a **4-liter jug** and a **3-liter jug**, measure exactly **2 liters** in one of the jugs.

Lab 3 - Task

Task 1; Implement the Water Jug Problem using Depth-First Search (DFS) and print the rules applied at each step.

Lab 4

Advanced Search Techniques

N-Queens Problem: Search Strategies and Optimization

- **Problem Definition:**
  - The **N-Queens Problem** is a classic combinatorial problem where N queens must be placed on an N × N chessboard so that no two queens attack each other.
  - This means no two queens should share the same **row, column, or diagonal**.

- **Search Strategies for Solving N-Queens:**
  - **Backtracking:**
    - A brute-force approach that places queens one by one and backtracks when a conflict arises.
  - **Hill Climbing (Local Search):**
    - Places all queens randomly and iteratively move towards a better state by reducing conflicts.
  - **Genetic Algorithm:**
    - Use evolutionary principles like selection, mutation, and crossover to find an optimal placement.

Introduction to Genetic Algorithms (GAs)

Genetic Algorithms (GAs) are **optimization techniques** inspired by the **process of natural selection and evolution**. They are used to find **optimal or near-optimal solutions** to complex problems where traditional methods might fail.

1. What Are Genetic Algorithms?

Genetic Algorithms (GAs) are a type of **search and optimization algorithm** that mimics the principles of **biological evolution**. They are used to solve problems by **evolving** solutions over multiple generations using principles like:

- **Selection** (Survival of the fittest)
- **Crossover** (Recombination of traits)
- **Mutation** (Random changes for diversity)

Why Use GAs?

- They **don't require specific formulas** to solve a problem.
- They can find solutions for **highly complex and non-linear** problems.
- They work well in **large search spaces** where brute-force methods fail.

2. How Do Genetic Algorithms Work?

GAs operate in a **cycle of evolution**, just like nature. Here are the key steps:

Step 1: Initialization (Creating a Population)

- A **population** is a group of possible solutions (individuals).
- Each **individual** (solution) is encoded as a **chromosome** (usually a string of numbers or binary values).

Example: Solving a **mathematical equation**, where each individual is a possible set of values for the equation variables.

Step 2: Fitness Function (Evaluating Solutions)

- Each individual is evaluated using a **fitness function** to determine **how good** it is as a solution.
- Higher fitness values mean better solutions.

Example Fitness Function:

For an equation **$f(x) = x^2$**, our goal is to **maximize f(x)**.

The **fitness of an individual** (e.g., x = 3) would be:

Fitness = (3)^2 = 9

The higher the fitness, the **better** the solution.

Step 3: Selection (Choosing the Best Parents)

- The best solutions (individuals) are **selected** to create the next generation.
- **Better solutions have a higher chance of being chosen**, similar to how stronger organisms survive in nature.

Common Selection Methods:

1. **Roulette Wheel Selection:** Probability-based selection based on fitness.
2. **Tournament Selection:** Choose the best from a random group.
3. **Rank-Based Selection:** Assign ranks and select proportionally.

Step 4: Crossover (Reproduction - Combining Parents)

- Two parents are **combined** to create a **new solution (offspring)**.
- This allows new individuals to **inherit good traits** from parents.

Example - One-Point Crossover:

Parent 1:	***1010 | 1101***
Parent 2:	***0110 | 1010***
Offspring 1:	***1010 | 1010***
Offspring 2:	***0110 | 1101***

Step 5: Mutation (Adding Diversity)

- Some individuals are **randomly changed** to maintain diversity and avoid getting stuck in local optima.
- Mutation ensures the algorithm explores new solutions.

Example of Mutation:

Original: ***10101101***
Mutated: ***10111101*** *(Bit 5 changed randomly)*

Step 6: Termination (Stopping Condition)

- The algorithm stops when:
  - A **satisfactory solution** is found.
  - A **maximum number of generations** is reached.

Final Output: The best individual (optimal solution) found so far.

3. Example Problems Solved by Genetic Algorithms

Example 1: Solving the Equation: $x^2 - 4x + 4 = 0$

Step 1: Define Population

Let's assume possible values of x are encoded as **binary chromosomes** (e.g., 0001 = 1, 0010 = 2).

Step 2: Define Fitness Function

The function we want to **minimize** is:

$$Fitness(x) = |x^2 - 4x + 4|$$

Lower values are better (closer to zero).

Step 3-6: Apply GA Steps

- **Selection:** Choose values of x with the lowest fitness scores.
- **Crossover:** Combine bits of selected chromosomes.
- **Mutation:** Randomly flip bits.
- **Termination:** Stop when fitness reaches 0 (best x found).

Output: $\boldsymbol{x = 2}$, *since* $2^2 - 4(2) + 4 = 0$.

Example 2: The Traveling Salesman Problem (TSP)

Problem: A salesman must visit **N** cities with the shortest route possible.

GA Solution:

- **Chromosome =** *Order of Cities (e.g., [A, C, B, D])*
- **Fitness Function =** *Total Distance Traveled*
- **Selection:** Keep shorter routes.
- **Crossover:** Swap cities between routes.
- **Mutation:** Randomly swap two cities.

GA Finds the Best Order of Cities to Minimize Travel Distance

Example 3: N-Queens Problem

Problem: Place **N queens** on an **N×N chessboard** so that no two queens attack each other.

GA Solution:

- **Chromosome =** *Row positions of queens (e.g., [2, 4, 1, 3])*
- **Fitness Function =** *Number of non-attacking pairs*
- **Mutation:** Swap two queen positions.
- **Crossover:** Mix two solutions.

GA finds a correct board setup

4. Applications for Genetic Algorithms

1. **Optimization Problems**
   - Pathfinding (e.g., Traveling Salesman Problem)
   - Scheduling (e.g., Timetables, Factory Workflow)
   - Resource Allocation (e.g., Cost Minimization)
2. **Machine Learning & AI**
   - Hyperparameter tuning in neural networks
   - Feature selection for better model accuracy
3. **Engineering & Design**
   - Circuit design & layout optimization
   - Antenna design for spacecraft
4. **Medical & Bioinformatics**
   - Drug discovery
   - DNA sequencing optimization

Prolog: Logic Programming for AI

- **Prolog (Programming in Logic)** is a declarative programming language used for AI and symbolic reasoning.
- **Syntax and Structure:**
  - **Facts:** Represent known truths.
  - queen(1, 2). % Queen at row 1, column 2
  - **Rules:** Define relationships between facts.
  - safe(X, Y) :- not(attacks(X, Y)).
  - **Queries:** Ask questions to the knowledge base.
  - ?- queen(X, Y).

- **Prolog for AI Problem Solving:**
  - Used in **state space search**, **rule-based systems**, and **constraint satisfaction problems**.

Lab 4 - Task

Task 1; Implement the N-Queens Problem (Dynamic)

Lab 5

Prolog for AI

Advanced Prolog Concepts: Facts, Rules, and Queries

- **Facts:** Represent basic assertions about the world.

```
parent(john, alice).
parent(alice, bob).
```

- **Rules:** Define logical relationships.

```
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

- **Queries:** Retrieve information from the knowledge base.

```
?- grandparent(john, bob).  % Output: true
```

Recursion and Backtracking in Prolog

- **Recursion:** Solves problems by breaking them into smaller subproblems.

```
factorial(0, 1).
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.
```

Query:

```
?- factorial(5, X).  % Output: X = 120
```

- **Backtracking:** Prolog explores all possible solutions.

```
member(X, [X|_]).
member(X, [_|T]) :- member(X, T).
```

Query:

```
?- member(3, [1, 2, 3, 4]).  % Output: true
```

Introduction to OpenCV

- **OpenCV (Open-Source Computer Vision Library)** is used for image processing, computer vision, and machine learning applications.

Key OpenCV Topics & Functions

Topic	Functions
Reading & Writing Images	cv2.imread(), cv2.imwrite()
Displaying Images	cv2.imshow(), cv2.waitKey(), cv2.destroyAllWindows()
Resizing & Cropping	cv2.resize(), array slicing
Color Spaces	cv2.cvtColor()
Drawing Shapes	cv2.line(), cv2.rectangle(), cv2.circle(), cv2.putText()
Image Thresholding	cv2.threshold(), cv2.adaptiveThreshold()
Edge Detection	cv2.Canny()
Contours & Shape Detection	cv2.findContours(), cv2.drawContours()
Face Detection	cv2.CascadeClassifier()

Lab 5 - Task

Task 1; Implement the main OpenCV functions in a Jupyter Notebook (.ipynb) (main topics, can skip others)
(https://www.geeksforgeeks.org/opencv-python-tutorial/)

Lab 6

Prolog for Expert Systems

Building AI Systems Using Prolog

- **Expert Systems** in Prolog are rule-based AI systems that use logic to make decisions.
- **Components of an Expert System:**
  - **Knowledge Base** (Facts & Rules)
  - **Inference Engine** (Logic-based reasoning)
  - **User Interface** (Queries and Outputs)

Example: Simple Medical Diagnosis Expert System

```
symptom(john, fever).
symptom(john, cough).
symptom(john, headache).

diagnosis(Patient, flu) :-
   symptom(Patient, fever),
   symptom(Patient, cough),
   symptom(Patient, headache).

?- diagnosis(john, Disease).  % Output: Disease = flu
```

Prolog for Natural Language Processing (NLP)

- **Prolog is used for:**
  - Parsing sentences
  - Understanding grammar structures
  - Translating natural language into logic

Example: Sentence Structure Analysis

```
sentence --> noun_phrase, verb_phrase.
noun_phrase --> determiner, noun.
verb_phrase --> verb, noun_phrase.

determiner --> [the].
noun --> [cat]; [dog]; [ball].
verb --> [chases]; [sees].

?- phrase(sentence, [the, cat, chases, the, dog]). % Output: true
```

Applications of OpenCV

- **Computer Vision Tasks Using OpenCV:**
  - Object Detection (Herd Detection, Face Profiling)
  - Facial Feature Measurement
  - Custom AI Applications

Lab 6 - Tasks

Task 1; Animal Herd Detection (with Map Alert for Bonus Marks)

1. Detects groups of animals in an image/video.
2. Uses OpenCV + YOLO/HAAR Cascade for object detection.
3. Optionally, triggers **alerts on a map** (e.g., marking herd location).
4. Must have a front-end based on flask

Bonus Feature: Use **Any Free API (or OpenStreetMap)** to mark the detected location.

Or

Task 2; Face Profiling (With Feature Measurements)

- Detects and **measures** facial features (eyes, nose, mouth, jawline).
- Defines a profile about the person, could be based on "**16 Personality Types**"
- Can Use OpenCV **dlib** library for precise face landmark detection.
- Must have a front-end based on flask

Or

Task 3; "Your Own Choice" (Custom AI/Computer Vision Project)

- Create a unique **AI-powered application** using **OpenCV.**
- Must have a front-end based on flask
- Examples:
    - **Hand Gesture Recognition**
    - **Automatic License Plate Recognition**
    - **Object Counting in Videos**

Lab 7

Knowledge Representation in AI

- **Knowledge Representation (KR)** is a field in Artificial Intelligence (AI) that focuses on how knowledge can be **stored, structured, and used by computers** to make intelligent decisions.
- It allows machines to **reason, infer, and understand relationships** between different concepts.

Types of Knowledge in AI

1. **Declarative Knowledge:** Facts and information about the world.
    - Example: "Water is a liquid."
2. **Procedural Knowledge:** Step-by-step instructions to solve a problem.
    - Example: "To make tea, boil water, add tea leaves, and steep for 5 minutes."
3. **Semantic Knowledge:** Meaning and relationships between concepts.
    - Example: "A cat is a type of animal."
4. **Episodic Knowledge:** Past experiences and events.
    - Example: "I visited Paris in 2020."
5. **Metaknowledge:** Knowledge about knowledge.
    - Example: "I know that I don't know everything."

Methods of Knowledge Representation

1. Logical Representation

- Uses **logic-based rules** to describe knowledge.
- **Propositional Logic** and **First-Order Logic (FOL)** are common methods.

Example: Propositional Logic

- "If it is raining, then the ground is wet."
  raining -> wet_ground.

Example: First-Order Logic (FOL)

- "All humans are mortal."
  human(X) -> mortal(X).

2. Semantic Networks

- Uses **graphs** to represent relationships between concepts.
- **Nodes = Entities** (e.g., "Dog"), **Edges = Relationships** (e.g., "is a type of").

Example: Semantic Network

[Dog] ----> (is a) ----> [Animal]
[Dog] ----> (has) ----> [Four Legs]
[Dog] ----> (can) ----> [Bark]

- Used in **Natural Language Processing (NLP)** and **Ontology Modeling**.

3. Frames (Object-Oriented Representation)

- Represents **knowledge in structured "frames"** (like objects in Object-Oriented Programming).
- Each frame contains **attributes (slots) and values**.

Example: Frame-Based Representation of a Car

```
{
  "Car": {
    "Type": "Sedan",
    "Brand": "Toyota",
    "Color": "Red",
    "Wheels": 4
  }
}
```

- Used in **AI Chatbots and Recommendation Systems**.

4. Production Rules (If-Then Rules)

- Knowledge is represented in **IF-THEN** rules.
- Used in **Expert Systems**.

Example: Medical Diagnosis System

IF fever AND cough THEN flu.
IF sore_throat AND headache THEN cold.

5. Ontologies (Formal Knowledge Representation)

- A **structured way** to define relationships between concepts.
- Used in **Semantic Web, AI Assistants, and Search Engines**.

Example: Ontology in Healthcare

[Disease] ----> (is_a) ----> [Flu]
[Flu] ----> (has_symptom) ----> [Fever]
[Flu] ----> (treated_with) ----> [Paracetamol]

- Used in **Google's Knowledge Graph** and **IBM Watson**.

Knowledge Representation in Prolog

Prolog is a **logic-based AI language** used for **Knowledge Representation and Reasoning**.

1. Defining Facts and Rules

```
% Facts
human(socrates).
human(plato).
mortal(X) :- human(X).

% Query
?- mortal(socrates). % Output: true
```

2. Expert System Example (Animal Classification)

```
animal(tiger).
animal(eagle).
animal(dolphin).

mammal(X) :- animal(X), not(bird(X)).
bird(X) :- animal(X), has_feathers(X).

?- mammal(tiger). % Output: true
?- bird(eagle).   % Output: true
```

Integrating Knowledge Representation with APIs

- AI applications use **APIs** to fetch **real-world knowledge**.

Example: Weather API for Decision-Making

```
raining(london).
need_umbrella(X) :- raining(X).

?- need_umbrella(london). % Output: true
```

What is an API?

- API (**Application Programming Interface**) is a set of rules that allow different software applications to communicate.
- **Types of APIs:**
  - **Web APIs:** RESTful APIs (e.g., JSON, XML)
  - **Library APIs:** Functions provided by software libraries
  - **Operating System APIs:** Interfaces for system operations

How to Use an API?

- **Make a request to an API** using HTTP methods:
  - **GET**: Retrieve data
  - **POST**: Send data
  - **PUT**: Update data
  - **DELETE**: Remove data

Example: Using Python Requests Library

```
import requests

url = "https://api.example.com/data"
response = requests.get(url)

if response.status_code == 200:
    data = response.json()
    print(data)
else:
    print("Error fetching data")
```

What is an API Format?

- APIs return data in formats such as:
  - **JSON (JavaScript Object Notation) – Most common**

    ```
    {"name": "Alice", "age": 25, "city": "New York"}
    ```

  - **XML (Extensible Markup Language)**

    ```
    <person>
      <name>Alice</name>
      <age>25</age>
      <city>New York</city>
    </person>
    ```

Using an API in a Flask Web App

Steps to Integrate an API into Flask:

1. Install Flask
2. pip install flask requests
3. Create a Flask App
4. Fetch API Data
5. Display Data in HTML Front-End

Lab 7 - Tasks

Task 1; Create any of the following, can use Free API & develop its "Flask" based back-end only:

1. Weather - App
2. Cooking / Recipe - App
3. Spotify - App
4. News - App
5. Vehicle Info - App
6. Useless Info - App
7. Random Joke - App
8. Stock Price - App
9. NASA - App
10. Your Own Choice - App

Lab 8

Implementing AI Models in Prolog

Why Use Prolog for AI?

Prolog is a **logic-based** programming language that excels at:

- **Symbolic reasoning**
- **Knowledge representation**
- **Rule-based expert systems**
- **Automated decision-making**

Unlike Python (which is widely used for ML models), Prolog is ideal for problems that involve **logical inference, search, and constraint satisfaction**.

AI Techniques in Prolog

1. **Expert Systems**
   - Knowledge-based systems that **simulate human experts**.
   - Example: Medical diagnosis systems.
2. **State-Space Search**
   - Used in pathfinding, problem-solving (e.g., **Water Jug Problem**).
3. **Constraint Satisfaction Problems (CSPs)**
   - Solves problems like **Sudoku, N-Queens, and Scheduling**.
4. **Natural Language Processing (NLP)**
   - Prolog can be used to build **parsers and chatbots**.

Example 1: Expert System in Prolog (Simple Medical Diagnosis)

```
% Facts
symptom(john, fever).
symptom(john, cough).
symptom(john, sore_throat).

% Rules
disease(X, flu) :- symptom(X, fever), symptom(X, cough), symptom(X,
sore_throat).
disease(X, cold) :- symptom(X, cough), symptom(X, runny_nose).

% Query
?- disease(john, What).
% Output: What = flu.
```

Example 2: AI Search Algorithm (Depth-First Search in Prolog)

```
% Graph representation
edge(a, b).
edge(b, c).
edge(c, d).
edge(b, e).

% Depth-First Search
dfs(Start, Goal, Path) :- dfs_helper(Start, Goal, [Start], Path).

dfs_helper(Goal, Goal, Path, Path).
dfs_helper(Node, Goal, Visited, Path) :-
    edge(Node, Next),
    \+ member(Next, Visited),
    dfs_helper(Next, Goal, [Next | Visited], Path).

% Query
?- dfs(a, d, Path).
% Output: Path = [d, c, b, a].
```

Propositional Logic in AI

What is Propositional Logic?

Propositional Logic (also called **Boolean Logic**) is a **formal system** in logic that uses **propositions (statements)** that can be **true (T) or false (F)**.

It is **widely used in AI** for:

- **Knowledge representation** (facts and rules).
- **Automated reasoning** (inference engines).
- **Decision-making systems** (expert systems).

Basic Components of Propositional Logic

1. **Propositions (Statements):**
   - **Atomic propositions:** Simple statements. Example:
     - *"It is raining."*
     - *"The sky is blue."*
   - **Compound propositions:** Formed by combining atomic propositions using **logical connectives**.
2. **Logical Connectives:**
   - **AND (∧)** → True if both propositions are true.
   - **OR (∨)** → True if at least one proposition is true.
   - **NOT (¬)** → Negates the truth value.
   - **IMPLICATION (→)** → "If P, then Q."
   - **BICONDITIONAL (↔)** → "P if and only if Q."

Examples of Propositional Logic Statements

Statement	Propositional Logic Expression
"It is raining and cold."	$R \wedge C$
"It is raining or cold."	$R \vee C$
"If it is raining, then the ground is wet."	$R \rightarrow W$
"The ground is wet if and only if it is raining."	$W \leftrightarrow R$
"It is not raining."	$\neg R$

Truth Tables in Propositional Logic

A **truth table** shows all possible values of a logical expression.

Example: AND (P ∧ Q)

P	Q	P ∧ Q
T	*T*	*T*
T	*F*	*F*
F	*T*	*F*
F	*F*	*F*

Example: OR (P ∨ Q)

P	Q	P ∨ Q
T	*T*	*T*
T	*F*	*T*
F	*T*	*T*
F	*F*	*F*

Propositional Logic in Prolog

Prolog can be used to **implement propositional logic** using **facts and rules**.

Example: Defining Propositional Logic in Prolog

```
% Facts
raining.
cold.

% Rules
wet_ground :- raining.
wear_jacket :- cold.

% Query
?- wet_ground.   % Output: true
?- wear_jacket.  % Output: true
```

Logic Programming in AI

What is Logic Programming?

- **Logic Programming** is a programming paradigm based on **formal logic**.
- A program consists of **facts**, **rules**, and **queries**.
- Prolog (Programming in Logic) is the most used **logic programming language**.

Key Concepts of Logic Programming

1. **Facts:**
   - Basic statements that are always true.
   - Example:

```
cat(whiskers).
dog(buddy).
```

2. **Rules:**
   - Derived facts based on conditions.
   - Example:

```
mammal(X) :- cat(X).
mammal(X) :- dog(X).
```

3. **Queries:**
   - Asking the system for information.
   - Example:

```
?- mammal(whiskers).  % Output: true
?- mammal(buddy).     % Output: true
```

Example: Family Tree in Prolog

```
% Facts
parent(john, mary).
parent(mary, alice).

% Rules
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).

% Queries
?- grandparent(john, alice).  % Output: true
```

Hybrid AI Systems: Integrating Prolog with Python

Why Integrate Prolog with Python?

- Prolog is great for **logic and reasoning**, while Python is excellent for **data handling and AI models**.
- Combining them allows us to build **hybrid AI systems** that **leverage logic and machine learning**.

Method 1: Using PySwip (Python Interface for Prolog)

1. Install PySwip

```
pip install pyswip
```

2. Python Code to Run Prolog Queries

```
from pyswip import Prolog

prolog = Prolog()
prolog.assertz("parent(john, mary)")
prolog.assertz("parent(mary, alice)")

result = list(prolog.query("parent(john, X)"))
print(result)  # Output: [{'X': 'mary'}]
```

Method 2: Calling Python from Prolog

- Prolog can **call Python functions** using external interfaces.

Example: Calling a Python Function from Prolog

```
:- use_module(library(python)).

run_python_function :-
    python_call(print("Hello from Python!")).
```

Lab 8 - Tasks

Task 1; Create any of the following, can use Free API & develop its "Flask" based back-end + font-end (make it dynamic aswell):

1. Weather - App
2. Cooking / Recipe - App
3. Spotify - App
4. News - App
5. Vehicle Info - App
6. Useless Info - App
7. Random Joke - App
8. Stock Price - App
9. NASA - App
10. Your Own Choice - App

Lab 9

Basics of NLP:

Natural Language Processing (NLP) is a field of AI focused on the interaction between computers and human (natural) languages. NLP helps machines understand, interpret, and generate human language.

Key Concepts:

- **Tokenization**:
  Breaks a sentence or paragraph into individual words or tokens.
  Example:
  sentence -> ["The", "dog", "barked"]
- **Stopwords**:
  Commonly used words like “is”, “the”, “in” that do not add significant meaning.
  These are usually filtered out in NLP tasks.
- **Stemming**:
  Reduces a word to its base/root form (may not be an actual word).
  Example: running -> run, played -> play
- **Lemmatization**:
  Similar to stemming but returns a valid word using grammar rules.
  Example: running -> run, better -> good

Use Cases of NLP

- Sentiment Analysis
- Chatbots and Virtual Assistants
- Machine Translation
- Spam Detection
- Text Classification and Summarization

Sentiment Analysis and Text Classification

Sentiment Analysis is the process of identifying emotions in text (positive, negative, or neutral).

Text Classification assigns predefined labels to text, e.g., spam or not spam, topic categories, etc.

Example: Sentiment Analysis using TextBlob

```
from textblob import TextBlob
text = TextBlob("This product is excellent!")
print(text.sentiment)
# Output: Sentiment(polarity=1.0, subjectivity=0.75)
```

Example: Text Classification using Sklearn

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB()
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(["I love it", "Hate this movie", "Amazing
experience"])
y = ["positive", "negative", "positive"]

model.fit(X, y)
print(model.predict(vectorizer.transform(["I hate it"])))
# Output: ['negative']
```

Lab 9 - Tasks

Task 1; Submit any task from the given subtasks within the Section: Natural Language Processing Tasks
(https://www.geeksforgeeks.org/natural-language-processing-nlp-tutorial)

Lab 10

Advanced NLP Concepts:

Traditional NLP models treat text as a bag of words, which ignores the order of words. **Sequence Models** (like RNNs, LSTMs, and Transformers) consider the **order and context** of words, making them ideal for tasks like translation, chatbots, and text generation.

Word Embeddings (Word2Vec)

Word Embeddings represent words in **dense vector space**, capturing their meanings and relationships. **Word2Vec** is a popular technique that uses a neural network to learn these vector representations. Words that are similar in meaning will have vectors that are close together in the embedding space.

Example:

```
from gensim.models import Word2Vec
sentences = [["king", "queen", "man", "woman"], ["paris", "france", "berlin", "germany"]]
model = Word2Vec(sentences, min_count=1)

print(model.wv.most_similar("king"))
```

Introduction to Artificial Neural Networks (ANNs)

ANNs are computing systems inspired by the biological neural networks of the brain. Composed of **layers of nodes (neurons)**:

- **Input Layer**: Takes input data
- **Hidden Layers**: Perform computation
- **Output Layer**: Produces result

ANNs are the foundation of **deep learning**, used for tasks such as:

- Image & speech recognition
- Natural language processing
- Predictive analytics

Simple ANN in Python (Keras):

```
from keras.models import Sequential
from keras.layers import Dense

model = Sequential()
model.add(Dense(10, input_dim=4, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Hands-on: Building a Simple Chatbot or Sentiment Analysis App

You can now combine your knowledge of **NLP, Word Embeddings, and ANNs** to build smart apps such as:

- Chatbots
- Sentiment Classifiers
- Question Answering systems

These apps can be developed using Flask (for backend) and HTML/CSS/JS (for frontend).

Lab 10 - Tasks

Task 1; Create a **chatbot** web application using **Flask** for backend and HTML/CSS for the frontend:

1. **University Admission Chatbot**
   Provides admission requirements, deadlines, and program info.

2. **Medical Center Information Bot**
   Answers questions about appointments, departments, and doctors.

3. **Hospital Information Chatbot**
   Responds with emergency services, room availability, visiting hours.

4. **Hotel Information Chatbot**
   Replies with room types, prices, amenities, and booking info.

5. **Restaurant Information Bot**
   Provides menus, reservation status, or order tracking.

6. **Library Assistant Bot**
   Helps with finding books, due dates, opening hours.

7. **Your Own Choice**
   Get creative! Build any intelligent chatbot (e.g., Mental Health Bot, Virtual Shopping Assistant, Campus Tour Bot)

Lab 11

Overview of Generative AI:

Generative AI refers to systems that can create new content such as text, images, music, or even code. These systems **learn patterns** from data and use them to generate new, similar content.

Common Generative AI Models:

- **GANs (Generative Adversarial Networks)**:
  Two neural networks (Generator & Discriminator) compete against each other to produce realistic data.
- **Diffusion Models**:
  Generate data by learning to reverse a gradual process of adding noise. Very effective for **image generation** (e.g., Stable Diffusion).
- **Transformers & LLMs**:
  Language models that generate human-like text, code, and conversations (e.g., GPT, BERT, T5).

What is GenAI?

GenAI (Generative AI) is the **branch of AI focused on content creation** rather than just decision-making or classification. Examples include ChatGPT (text), DALL·E (images), and Codex (code generation).

What is LLM (Large Language Model)?

LLMs are deep learning models trained on massive datasets of human language. Capable of:

- Language understanding & generation
- Summarization
- Translation
- Sentiment analysis
- Chatbots and virtual assistants

Examples: GPT-4, PaLM, LLaMA, Falcon

What is RAG (Retrieval-Augmented Generation)?

RAG = LLM + Search

Combines language generation with real-time document retrieval. Allows an LLM to **retrieve facts** from external sources (like a database or PDF) before answering a query.

Use Case: Chatbots that pull live information from company documents.

What is FAISS?

Facebook AI Similarity Search (FAISS) is a library for **efficient similarity search** and clustering of dense vectors. It's used to **search through embeddings** quickly (e.g., finding similar documents or questions).

What is a Vector in AI?

A **vector** is a list of numbers that represents **meaningful data** (like a word, image, or sentence) in numeric form. These are often created using models like BERT, Word2Vec, or CLIP.

Example:

"apple" → [0.21, -0.34, 0.76, ..., 0.02]

These numbers represent semantic meaning.

What is a VectorDB (Vector Database)?

A **Vector Database** stores and indexes vectors (embeddings) for **fast similarity search**. Useful in GenAI apps where you want to **retrieve similar documents or questions**.

Popular examples: Pinecone, Weaviate, ChromaDB, Qdrant

What is Hugging Face?

Hugging Face is a **platform and library** for working with NLP and AI models, especially **LLMs and Transformers**. Features:

- Transformers library (BERT, GPT, T5, etc.)
- Pre-trained models and datasets
- Model training, hosting, and inference APIs

Hugging Face makes it **easy to access, fine-tune, and deploy AI models** in research and production.

LLMs and Their Applications

- Language generation (text completion, story writing)
- Virtual Assistants and Chatbots
- Summarization and Translation
- Coding assistants
- Automated customer support
- Question Answering Systems

Hugging Face Framework for Generative AI

Hugging Face offers:

- transformers – A Python library to load and use pretrained models
- datasets – Ready-to-use NLP datasets
- accelerate – For model training at scale
- spaces – To deploy your GenAI apps (like mini web apps)

Example:

```
from transformers import pipeline
chatbot = pipeline("text-generation", model="gpt2")
print(chatbot("Hello, I'm a chatbot built with Hugging Face!"))
```

Lab 11 - Tasks

Task 1; Describe the Difference Between the Following Concepts:

1. **LangChain**
2. **LLMs (**Large Language Models**)**
3. **RAG (**Retrieval-Augmented Generation**)**
4. **FAISS (**Facebook AI for Semantic Search**)**
5. **Vector**
6. **VectorDB**
7. **Generative AI (**GenAI**)**
8. **GANs (**Generative Adversarial Networks)

Lab 12

Hands-on:
Working with OpenAI GPT APIs and Hugging Face Model

OpenAI and Hugging Face provide powerful APIs and pre-trained models that enable developers to build advanced NLP and QnA applications. These APIs allow integration of **LLMs**, **chatbots**, **QnA systems**, and more using Python or web frameworks like Flask.

OpenAI GPT API (Overview):

Used to access models like GPT-3.5 and GPT-4. • Can be used for:

- Text generation
- Question answering
- Summarization
- Chat-based applications

Example (Python):

```
import openai

openai.api_key = 'your_api_key_here'

response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": "What is Artificial Intelligence?"}]
)
print(response.choices[0].message['content'])
```

Hands-on:
Developing QnA Hadith Bot using Hugging Face MiniLM Model

Objective:
Develop a **Hadith Question-Answering Bot** using the **MiniLM model** from Hugging Face. The system embeds Hadith data, stores it using **FAISS** (a Vector Similarity Search engine), and performs retrieval based on user queries.

Steps to Build Hadith QnA Bot:

1. Data Loading & Preprocessing:

- **Dataset Used:** LK Hadith Corpus – contains multiple CSV files of Hadith data in Arabic and English.
- **Steps:**
  - All CSVs are loaded using glob.
  - Columns include English_Hadith, Arabic_Hadith, Chapter, Section, Grade, etc.
  - Null or empty entries are removed.
  - Each hadith text is cleaned (lowercased, removed symbols) using regex.

```
def clean_text(text):
    if isinstance(text, str):
        text = re.sub(r'[^A-Za-z\s]', '', text)
        text = text.lower()
    else:
        text = ''
    return text
```

2. Sentence Embedding using Hugging Face (MiniLM)

- English embeddings: 'paraphrase-MiniLM-L6-v2'
- Arabic embeddings: 'bert-base-multilingual-cased'
- Embeddings are stored as dense vectors (numpy arrays).

```
from sentence_transformers import SentenceTransformer

model_en = SentenceTransformer('paraphrase-MiniLM-L6-v2')
embeddings_en = model_en.encode(hadith_df_en['Hadith'].tolist())
```

3. Vector Indexing using FAISS

- FAISS (Facebook AI Similarity Search) helps efficiently store and retrieve dense vectors.
- The bot uses IndexFlatL2 to store embeddings (Euclidean distance based).
- Index is saved using .write_index() for reuse.

```
import faiss

d = embeddings_en.shape[1]
index = faiss.IndexFlatL2(d)
index.add(embeddings_en)
faiss.write_index(index, 'hadith_faiss_en.index')
```

4. Search / Retrieval Function:

- Takes a **user query**, embeds it using MiniLM, and searches the FAISS index.
- Returns the top 5 similar Hadiths with distances.

```
def retrieve_similar_hadiths(query, model, index, hadith_df, k=5):
    query_embedding = model.encode([clean_text(query)])
    distances, indices = index.search(query_embedding, k)

    for i in range(k):
        print(f"Hadith {i+1}:")
        print(hadith_df['Hadith'].iloc[indices[0][i]])
        print(f"Distance: {distances[0][i]}\n")
```

5. Sample Queries:

```
query = "Fasting in Ramadan"
retrieve_similar_hadiths(query, model, index, hadith_df)

query = "Signs of the End of Time"
retrieve_similar_hadiths(query, model, index, hadith_df)
```

Outputs:

- Top 5 Hadiths related to the query are displayed with similarity distances.
- Very relevant Hadiths are returned based on semantic meaning.

Lab 12 - Tasks

Task 1;

1. **Run the HadithBot.ipynb** file (as discussed in class).
2. **Then create your own QnA Bot** from the topic you selected in **Lab 10**, using the same pipeline:
    - Preprocess your dataset (text-based QnA data)
    - Embed questions/answers using Hugging Face MiniLM
    - Store vectors using FAISS
    - Search and match using similarity
    - Display results via a Flask + HTML UI

Lab 13

Lab 13 - Tasks

Task 1;

1. **Project Submission:**
    - Project can be based on NLP + Flask
    - Project can be based on OpenCV + Flask
    - Project can be based on GenAI + Flask

Lab 14

Lab 14 - Tasks

Task 1;

1. **Project Demo Submission (Video):**
    - Submit the Video of Project's Demo

Lab 15

Lab 15 - Tasks